Manual Setup Guide

Azure Bot Service + Microsoft Agent Framework + Azure GPT-4o

A practical, end-to-end walkthrough for creating the cloud resources, configuring the Python weather agent, exposing it through Dev Tunnel, and testing it in Azure Web Chat.

SOLUTION	THIS WORKSPACE
Azure Bot	guanghuiBotHandle01
Azure AI Services	guazhaAgentFramework01
Foundry project	guazhaAgentFramework01_Proj01
Model deployment	gpt-4o
Local application	agent-framework

Prepared for local development on Windows with Python 3.12

Version 1.0 - September 9, 2026

Contents and success criteria

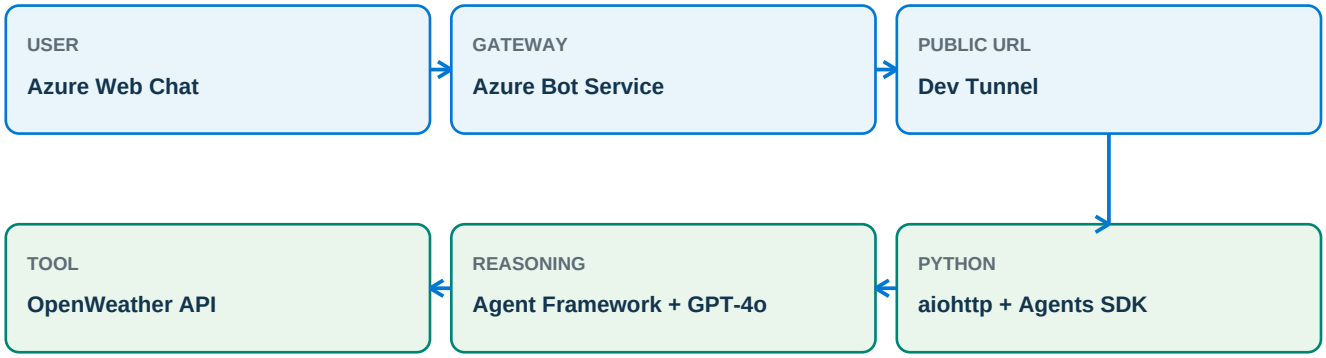
1. Architecture and resource map
2. Prerequisites and security preparation
3. Create Microsoft Foundry resources and deploy GPT-4o
4. Create the Azure Bot and its Entra application identity
5. Obtain an OpenWeather API key
6. Prepare the Python project and environment
7. Configure .env safely
8. Run and validate the local agent
9. Create a Dev Tunnel and configure the messaging endpoint
10. Test with Azure Web Chat and Agents Playground
11. Understand the runtime message path
12. Troubleshooting and production hardening

Definition of done
Azure Web Chat sends a message to the public Dev Tunnel URL; the local Python handler receives it; GPT-4o selects a weather tool; OpenWeather returns data; and the formatted response appears in Web Chat.

Security rule
Never place a real client secret or API key in screenshots, chat, source control, or this PDF. Values shown below use placeholders. Store local secrets only in **.env**, which must be ignored by Git.

1. Architecture and resource map

Understand the boundaries before configuring them.



Component	Role	Authentication
Azure Web Chat	Browser interface where the user types messages.	Direct Line/Web Chat channel token
Azure Bot Service	Routes Bot Framework activities between channels and your endpoint.	Bot App ID and Entra identity
Dev Tunnel	Maps a public HTTPS URL to localhost:3978 during development.	Anonymous tunnel access; bot traffic still uses JWT
aiohttp + Agents SDK	Receives POST /api/messages, validates JWT, dispatches activities, sends replies.	Azure Bot JWT and bot client credential
Agent Framework	Runs GPT-4o, maintains an AI session, and executes tools.	Azure OpenAI API key or Entra token
OpenWeather	Provides current conditions and five-day forecast data.	OpenWeather API key

Workspace-specific non-secret values

Setting	Value
Subscription	91d74870-4fcf-4e38-9599-819c6cacc9f0
Tenant	b0435f53-aeb2-434e-9765-c2548b755f65
Resource group	azureai-streaming
Bot App ID	552974da-683f-4909-8287-aeee13779439
Azure AI endpoint	https://guazhaagentframework01.cognitiveservices.azure.com/
Model deployment name	gpt-4o

2. Prerequisites and security preparation

Requirement	How to verify on Windows
Azure subscription access	Sign in to portal.azure.com and confirm you can create resources.
Python 3.10+	Run: py -0p
Git	Run: git --version
Azure CLI	Run: az --version
Dev Tunnel CLI	Run: devtunnel --version
Agents Playground	Optional. Install with winget install agentsplayground
OpenWeather account	Create an account at https://openweathermap.org/api

Sign in to the correct Azure tenant

```
az login --tenant MngEnvMCAP362879.onmicrosoft.com --use-device-code
az account set --subscription 91d74870-4fcf-4e38-9599-819c6cacc9f0
az account show --output table
```

Check the account before every resource command
The active account must be in tenant **b0435f53-aeb2-434e-9765-c2548b755f65** and subscription **91d74870-4fcf-4e38-9599-819c6cacc9f0**. A cached account from another tenant causes SubscriptionNotFound errors.

Create a secret handling plan

1. **Use placeholders while building**
   Do not paste real values until the file is ignored by Git.
2. **Add .env to .gitignore**
   Also ignore .venv/, __pycache__/, and *.pyc.
3. **Use short-lived development credentials**
   Create a separate credential with a clear name and expiration date.
4. **Rotate exposed values**
   If a secret appears in terminal output, chat, screenshots, or commits, revoke and replace it.

3. Create Foundry resources and deploy GPT-4o

Portal path: **ai.azure.com** - Create project - Models + endpoints

1. **Open Microsoft Foundry**
   Go to **https://ai.azure.com**, sign in, and select the target tenant and subscription.
2. **Create or open a project**
   Create a Foundry project and select an Azure AI Services resource. This setup uses project **guazhaAgentFramework01_Proj01** and resource **guazhaAgentFramework01**.
3. **Open Models + endpoints**
   In the project navigation, choose **Models + endpoints**, then select **Deploy model**.
4. **Deploy GPT-4o**
   Choose the **gpt-4o** model. Name the deployment **gpt-4o**. Wait for provisioning state **Succeeded**.
5. **Record the deployment name**
   The application uses the deployment name, not merely the catalog model family name.
6. **Record the resource endpoint**
   In Azure Portal, open **guazhaAgentFramework01 - Resource Management - Keys and Endpoint**. Copy the Endpoint field, but never copy a key into documentation.

```
AZURE_OPENAI_ENDPOINT=https://guazhaagentframework01.cognitiveservices.azure.com/
AZURE_OPENAI_MODEL=gpt-4o
```

Optional Azure CLI verification

```
az cognitiveservices account show --resource-group azureai-streaming --name guazhaAgentFramework01
--query properties.endpoint --output tsv
az cognitiveservices account deployment show --resource-group azureai-streaming --name
guazhaAgentFramework01 --deployment-name gpt-4o --output table
```

Endpoint shape
Use the parent Azure AI Services endpoint ending in **cognitiveservices.azure.com/**. Do not use the browser playground URL. The SDK combines the endpoint, deployment name, and API version.

4. Create the Azure Bot and Entra identity

Portal path: **Azure Portal - Create a resource - Azure Bot**

1. **Create the bot resource**
   Choose subscription, resource group, bot handle, pricing tier, and Microsoft App ID type. For this setup: resource group **azureai-streaming**, handle **guanghuiBotHandle01**, type **Single Tenant**.
2. **Create or select an App Registration**
   The Azure Bot uses a Microsoft Entra application identity. Record the Application (client) ID and Directory (tenant) ID.
3. **Create a client secret**
   Open **Microsoft Entra ID - App registrations - your app - Certificates & secrets**. Select **New client secret**. Give it a descriptive name and short expiration.
4. **Copy only the secret Value**
   The portal shows the secret value once. The Secret ID is not the credential. Store the value securely.
5. **Confirm bot identity settings**
   Open **guanghuiBotHandle01 - Settings - Configuration**. Confirm Microsoft App ID, App Type, and Tenant ID.
6. **Leave Messaging endpoint for later**
   You will set this after Dev Tunnel creates the public HTTPS URL.

Bot setting	Workspace value / action
Microsoft App ID	552974da-683f-4909-8287-aeee13779439
App type	Single Tenant
Tenant ID	b0435f53-aeb2-434e-9765-c2548b755f65
Client secret	Create manually; keep out of source control and documentation
Messaging endpoint	Set after Step 9 to https://TUNNEL_HOST/api/messages

5. Obtain an OpenWeather API key

1. **Create an OpenWeather account**
   Go to **https://openweathermap.org/api**, register, and verify the account.
2. **Open API keys**
   Open your profile and select **My API keys**. Create a key with a recognizable development name.
3. **Allow activation time**
   A new key may return HTTP 401 until activation completes. This can take time even when the key is displayed.
4. **Store it only in .env**
   Set **OPEN_WEATHER_API_KEY**. Never put the key in Python source or log the full request parameters.
5. **Validate the API**
   Request current weather for a known city. HTTP 200 means the key is active; HTTP 401 means invalid or not yet activated.

```
https://api.openweathermap.org/data/2.5/weather?q=Washington,DC,US&appid=<OPEN_WEATHER_API_KEY>&units=imperial
```

> **Logging warning**
> Do not log the query dictionary when it contains **appid**. Log only the city, state, country, HTTP status, and a redacted error message.

6. Prepare the Python project

Run these commands from the workspace root in PowerShell.

Option A - Download only the sample with Git sparse checkout

```
git clone --depth 1 --filter=blob:none --sparse https://github.com/microsoft/Agents.git
microsoft-agents-temp
git -C microsoft-agents-temp sparse-checkout set samples/python/agent-framework
Copy-Item microsoft-agents-temp/samples/python/agent-framework/* agent-framework -Recurse
Remove-Item microsoft-agents-temp -Recurse -Force
```

Create an isolated Python 3.12 environment

```
cd agent-framework
py -3.12 -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

This workspace also uses Azure Identity for keyless local GPT authentication:

```
python -m pip install azure-identity
```

Required project files

File	Responsibility
src/main.py	Application entry point and Agents SDK logging.
src/start_server.py	aiohttp host, JWT middleware, and POST /api/messages route.
src/agent.py	Bot adapter, authentication, GPT client, agent instructions, handlers, state, streaming.
src/tools/date_time.py	Current date/time tool exposed to GPT-4o.
src/tools/weather_lookup.py	Current weather and forecast tools using OpenWeather.
.env	Local configuration and secrets. Never commit.

7. Configure .env safely

Create **.env** from **env.TEMPLATE**. Use the following structure and replace only placeholders.

```
CONNECTIONS__SERVICE_CONNECTION__SETTINGS__CLIENTID=552974da-683f-4909-8287-aeee13779439
CONNECTIONS__SERVICE_CONNECTION__SETTINGS__CLIENTSECRET=<BOT_CLIENT_SECRET_VALUE>
CONNECTIONS__SERVICE_CONNECTION__SETTINGS__TENANTID=b0435f53-aeb2-434e-9765-c2548b755f65

AZURE_OPENAI_ENDPOINT=https://guazhaagentframework01.cognitiveservices.azure.com/
AZURE_OPENAI_API_KEY=
AZURE_OPENAI_MODEL=gpt-4o
AZURE_OPENAI_API_VERSION=2024-10-21

OPEN_WEATHER_API_KEY=<OPENWEATHER_API_KEY>
```

Authentication choices for GPT-4o

Mode	Configuration	Recommended use
Azure OpenAI key	Set AZURE_OPENAI_API_KEY.	Simple local test; protect and rotate the key.
Azure CLI credential	Leave AZURE_OPENAI_API_KEY blank; run az login in the correct tenant.	Local development without storing a model key.
Managed identity	Use DefaultAzureCredential or ManagedIdentityCredential and assign an Azure OpenAI role.	Production deployment.

Create .gitignore before entering secrets

```
.env
.venv/
__pycache__/
*.py[cod]
```

Restart required
The application reads .env during Python module import. After changing any value, stop and restart **python -m src.main**. Existing processes do not reload the file.

8. Run and validate the local agent

1. **Activate the environment**
   Open PowerShell in the **agent-framework** folder and run **.venv\Scripts\Activate.ps1**.
2. **Check dependencies**
   Run **python -m pip check**. Expect: No broken requirements found.
3. **Compile source**
   Run **python -m compileall -q src**. No output means success.
4. **Start the server**
   Run **python -m src.main**. Expect: Running on http://localhost:3978.
5. **Probe authentication**
   An empty unauthenticated POST should return HTTP 401 on the authenticated bot server. This proves the route and JWT middleware are active.
6. **Test GPT-4o independently**
   Run a minimal Agent Framework request. A short response proves endpoint, deployment, API mode, and credentials.
7. **Test OpenWeather independently**
   Load .env, call get_current_weather for Washington, DC, and confirm factual output.

```
python -m pip check
python -m compileall -q src
python -m src.main
```

Keep this terminal open
The Python process is the bot backend. Closing it stops message processing even if Azure Bot and Dev Tunnel still exist.

9. Dev Tunnel and Azure Bot messaging endpoint

Azure Bot cannot call localhost directly. Dev Tunnel supplies a temporary public HTTPS endpoint.

1. **Install Dev Tunnel**
   Install the Microsoft Dev Tunnel CLI if **devtunnel --version** is unavailable.
2. **Sign in**
   Run **devtunnel user login** if needed, then confirm with **devtunnel user show**.
3. **Host local port 3978**
   Run **devtunnel host -p 3978 --allow-anonymous**. Keep this terminal open.
4. **Copy the browser URL**
   Copy the value shown after **Connect via browser:**. It resembles https://<id>-3978.<region>.devtunnels.ms.
5. **Open Bot Configuration**
   Azure Portal - **guanghuiBotHandle01 - Settings - Configuration**.
6. **Set Messaging endpoint**
   Enter **https://<tunnel-host>/api/messages** and select **Apply** or **Save**.
7. **Verify the saved value**
   Reload Configuration and confirm the complete URL remains in the Messaging endpoint field.

```
devtunnel host -p 3978 --allow-anonymous

# Example shape only; use the URL printed by your tunnel
https://<tunnel-host>/api/messages
```

Optional Azure CLI equivalent

```
az bot update --resource-group azureai-streaming --name guanghuiBotHandle01 --endpoint
"https://<tunnel-host>/api/messages"
```

Why /api/messages is required
The public hostname reaches port 3978, while **/api/messages** matches the aiohttp route registered in **src/start_server.py**. Omitting the path produces a 404 or never reaches the handler.

10. Test the complete system

Azure Web Chat

1. **Confirm both processes are running**
   Terminal A: **python -m src.main**. Terminal B: **devtunnel host ...**.
2. **Open Test in Web Chat**
   Azure Portal - **guanghuiBotHandle01 - Settings - Test in Web Chat**.
3. **Wait for Connected**
   The chat panel should show Connectivity Status: Connected and the welcome message.
4. **Send a deterministic test**
   Type: **What is the current weather in Washington, D.C.?**
5. **Observe each layer**
   Web Chat shows typing; Python logs the incoming activity and weather location; the final response contains temperature, humidity, wind, and conditions.

Agents Playground

```
winget install agentsplayground
agentsplayground
```

Use **http://localhost:3978/api/messages** and configure the bot App ID, tenant ID, and client secret when authentication is enabled. For an isolated anonymous local test, run a second instance with a different PORT and ANONYMOUS_ALLOWED=true.

> **Expected successful weather response**
> The exact values change, but the answer should include current temperature, low, high, humidity, wind speed, and a condition description. A generic apology usually means the tool returned an error or the handler caught an exception.

11. How a message moves through the code

Stage	What happens	Code or service
1	User sends text in Web Chat.	Azure Web Chat
2	Azure Bot wraps text in a Bot Framework Activity and POSTs it with a JWT.	Azure Bot Service
3	Public HTTPS request is forwarded to localhost:3978/api/messages.	Dev Tunnel
4	JWT middleware validates the request; start_agent_process parses the Activity.	src/start_server.py
5	AgentApplication dispatches activity type message to on_message.	src/agent.py
6	Handler reads context.activity.text and retrieves the conversation session.	on_message
7	Agent Framework sends instructions, history, message, and tool schemas to GPT-4o.	WEATHER_AGENT.run
8	GPT-4o selects get_current_weather and supplies structured arguments.	Agent Framework
9	Python calls OpenWeather and returns factual text to GPT-4o.	weather_lookup.py
10	GPT-4o formats the final answer; chunks enter streaming_response.	on_message
11	CloudAdapter authenticates and sends reply activities to Bot Framework.	Agents SDK + MSAL
12	Azure Bot delivers the reply to the Web Chat browser.	Azure Bot Service

The central handler

```
@AGENT_APP.activity("message")
async def on_message(context: TurnContext, state: TurnState):
user_text = (context.activity.text or "").strip()
session_data = state.get_value("ConversationState.agentSession", lambda: None)
if session_data is None:
session_data = WEATHER_AGENT.create_session()
async for chunk in WEATHER_AGENT.run(user_text, session=session_data, stream=True):
if chunk.text:
context.streaming_response.queue_text_chunk(chunk.text)
state.set_value("ConversationState.agentSession", session_data)
await context.streaming_response.end_stream()
```

12. Troubleshooting

Symptom	Likely cause	Check and fix
SubscriptionNotFound	Azure CLI is using another tenant or subscription.	Run az account list --all, then az account set --subscription .
HTTP 401 from /api/messages	Expected for an unauthenticated probe, or bot credentials/JWT mismatch.	For Web Chat, verify App ID, tenant, client secret, and bot app type.
Web Chat not connected	Python server or tunnel is stopped; endpoint is wrong.	Start both processes and verify Configuration - Messaging endpoint.
404 from GPT-4o	Wrong endpoint/API route or deployment does not support Responses API.	Use OpenAIChatCompletionClient and verify endpoint plus deployment name.
Azure OpenAI missing credentials	Neither API key nor Entra token provider is available.	Set AZURE_OPENAI_API_KEY or sign in and use an Azure credential.
OpenWeather HTTP 401	Invalid, inactive, or stale API key.	Test the key directly, wait for activation, update .env, then restart Python.
Tool says key not configured	.env was not loaded or variable name is wrong.	Run from project root, confirm load_dotenv(), spelling, and restart.
Old behavior after editing .env	Existing Python process retains old environment values.	Stop and restart every running server instance.
Message remains Sending	Tunnel delivery or backend processing is delayed/failing.	Check Dev Tunnel output, Python logs, and Web Chat Connectivity Status.
Lost conversation history	MemoryStorage was cleared by process restart.	Expected locally; use durable storage in production.

Diagnostic order

Check from the inside out: (1) tool API, (2) GPT-4o invocation, (3) local /api/messages host, (4) Dev Tunnel, (5) Azure Bot Configuration, (6) Web Chat. This isolates one boundary at a time.

13. Production hardening and cleanup

1. **Deploy the Python backend**
   Use Azure App Service, Azure Container Apps, or another durable HTTPS host. Configure startup as **python -m src.main** and bind the platform port.
2. **Replace Dev Tunnel**
   Set the bot Messaging endpoint to the permanent deployment URL plus **/api/messages**.
3. **Use managed identity**
   Replace AzureCliCredential with a production credential and grant the identity the minimum Azure OpenAI data-plane role.
4. **Move secrets out of .env**
   Use Azure application settings and Key Vault references. Do not deploy a local .env file.
5. **Use durable state**
   Replace MemoryStorage with shared storage so conversation history survives restarts and multiple instances.
6. **Add resilience**
   Set outbound HTTP timeouts, bounded retries, structured tool errors, health endpoints, and telemetry.
7. **Restrict access**
   Review Bot channels, networking, role assignments, secret expiration, and public access settings.
8. **Remove development credentials**
   Delete short-lived app secrets and rotate any value exposed during troubleshooting.

Final operational checklist

```
[ ] Bot endpoint uses permanent HTTPS and ends with /api/messages
[ ] No secrets exist in source control, PDF files, screenshots, or logs
[ ] Managed identity can invoke the GPT-4o deployment
[ ] OpenWeather key is stored in Key Vault or protected app settings
[ ] Conversation state is durable and shared
[ ] Logs include correlation IDs but redact credentials and tokens
[ ] Monitoring and alerts cover HTTP errors, model failures, and tool failures
[ ] Temporary Dev Tunnel and test credentials are removed
```

You are finished when
A clean deployment can be recreated using only this guide and securely supplied secrets; Web Chat reaches the deployed Python endpoint; GPT-4o can call tools; and all temporary local-development infrastructure has been removed.

Appendix A. Quick reference

Portal navigation

Task	Portal path
Find Azure AI endpoint	guazhaAgentFramework01 - Resource Management - Keys and Endpoint
Find model deployment	Microsoft Foundry project - Models + endpoints - gpt-4o
Configure bot endpoint	guanghuiBotHandle01 - Settings - Configuration - Messaging endpoint
Test bot	guanghuiBotHandle01 - Settings - Test in Web Chat
Create bot secret	Microsoft Entra ID - App registrations - bot app - Certificates & secrets
Inspect bot channels	guanghuiBotHandle01 - Settings - Channels

Local command sequence

```
# Terminal 1 - Python backend
cd agent-framework
.venv\Scripts\Activate.ps1
python -m src.main

# Terminal 2 - Public tunnel
devtunnel host -p 3978 --allow-anonymous

# Terminal 3 - Optional local UI
agentsplayground
```

Core URLs

Purpose	URL pattern
Local agent	http://localhost:3978/api/messages
Public development endpoint	https://TUNNEL_HOST/api/messages
Azure AI Services endpoint	https://guazhaagentframework01.cognitiveservices.azure.com/
OpenWeather current conditions	https://api.openweathermap.org/data/2.5/weather
OpenWeather forecast	https://api.openweathermap.org/data/2.5/forecast

End of guide